ChatGPT

Google ADK Runtime and Orchestration Overview

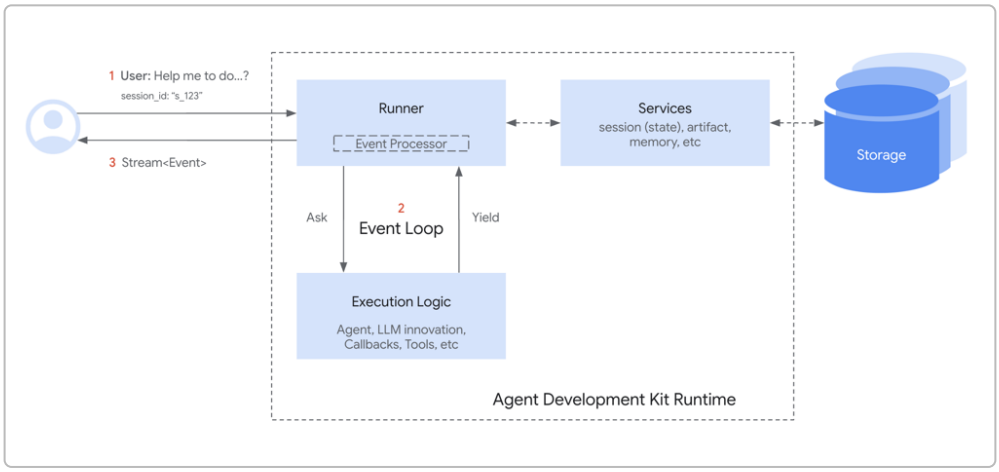

Illustration: The ADK runtime architecture. The Runner orchestrates an event loop between user input, the agent execution logic, and backend services (session state, artifact storage, memory). Events (messages, tool calls/results, etc.) flow between the Runner and agents, while persistent data is managed via services. [1] [2]

ADK Runner and the Event Loop

In Google's Agent Development Kit (ADK), the **Runner** is the central engine that brings agents to life. The `Runner` bridges your application (which provides user input) and the agent system, managing the entire **inference loop** for each query [3] [4]. When you initialize a Runner, you typically provide an `app_name` (namespacing your app's conversations), a root `agent` (the top-level agent to run), and the service implementations for session, artifact, and memory management [5]. For example, you might use an `InMemorySessionService` (which keeps conversation data in RAM), a `DatabaseSessionService` for persistent sessions, and an `InMemoryArtifactService` or `GcsArtifactService` for storing larger data artifacts [6] [7].

Runner's Responsibilities: Once set up, the Runner handles several key tasks during execution [8] [9]:

- **Session Management:** It works with the `BaseSessionService` to load or create a conversation session, including retrieving past events and state at the start of an interaction, and saving new events as the conversation progresses [10]. This ensures each conversation (session) has continuity in its context/history.
- **Agent Invocation:** It determines which agent to run – usually your root agent for a new conversation, or if resuming, the appropriate agent in context – and calls the agent's asynchronous run method (`run_async` in Python) [11]. Essentially, the Runner "kicks off" the agent logic for the user's query.
- **Context Creation:** The Runner constructs an **InvocationContext** for the agent, which bundles together the session info, services, user input, and run configuration [4]. This context (`ctx`) is

passed into the agent, giving it access to things like `ctx.session` (including conversation state and history) and the ability to call services if needed.

- **Event Streaming and Processing:** As the agent runs, it doesn't return a final answer all at once – instead, it yields a sequence of **Event** objects. The Runner continuously listens for these events in an **event loop** [12] [13]. For each `Event` yielded by the agent, the Runner immediately processes it: it appends the event to the session (via the SessionService) and applies any side-effects (like updating session state or saving an artifact) indicated by the event's content [14] [15]. After processing, the Runner forwards the event to the calling application or UI (so you can, for example, stream intermediate responses to the user) [12] [16]. It then signals the agent that the event was handled, allowing the agent to resume execution and produce the next event [17] [18].
- **Input Handling:** The Runner also takes care of initial user input and any special handling needed. For instance, when you pass a new user message to `runner.run_async(...)`, the Runner will first record that user message as an event in the session history [17]. If the input includes a binary blob (e.g. an image or audio file), the Runner can save it via the ArtifactService and instead pass a reference to the agent [9]. This prevents large payloads from bloating the event history – they'll be stored as artifacts and only references or IDs appear in events.

The Event Loop: Under the hood, the Runner orchestrates a loop of communication between itself and the agent (or "execution logic"). Think of it as a turn-based exchange [2] [19]:

1. The Runner receives the user's query (as a new event appended to the session) and calls the agent's `run_async` method to handle it [20] [21].
2. The agent (or one of its sub-agents) runs and may **yield an Event** whenever it needs to report something or request an action. For example, an LLM-based agent might yield an event to say "I want to call a tool now," or yield partial message content if streaming, etc.
3. The Runner catches that event, **processes side-effects** (updates state, stores artifacts, etc.) via the services, then yields the event up to the application/UI [14] [22]. Only after the event is fully processed and recorded does the Runner tell the agent "OK, continue."
4. The agent resumes from where it left off, now aware of any state changes or tool results that happened, and continues its logic until it yields the next event [15] [18].
5. Steps 3–4 repeat until the agent signals it's done with the user's query (e.g. by yielding a final response event and no further events). At that point, the Runner's loop ends for that invocation.

This event-driven loop is the core of ADK's **inference chain**, ensuring a clear order of operations: The agent decides and emits an event, the Runner locks in any changes from that event (making them persistent in the session) and delivers the output, then the agent can move on [23] [24]. This design guarantees that things like session state updates or tool outputs are applied at the correct time, preventing the agent from seeing uncommitted or inconsistent state. It also enables features like streaming responses (the Runner can send out partial events as they come) and tool usage (the Runner can pause the agent to execute tools and then resume it with results).

Notably, ADK allows you to customize runtime behavior via a `RunConfig` passed to `runner.run_async`. The `RunConfig` lets you toggle options such as streaming mode, speech recognition or synthesis, input blob handling, or limits on LLM calls [25] [26]. For example, you can enable partial `event.partial=True` chunks for real-time streaming, or set `save_input_blobs_as_artifacts=True` so that user uploads are stored through the ArtifactService automatically [27] [28]. These configurations fine-tune the Runner's operation but the core loop remains the same.

Sessions, State, and the SessionService

Every conversation or chat in ADK is tracked as a **Session**. A session is essentially the context for a dialogue – it has a unique `id` and ties together a sequence of interactions (events) along with any transient state data for that dialogue [29] [30]. Think of it as the conversation's memory. The session is identified by an `app_name` (which groups sessions by application/agent), a `user_id` (to scope sessions per end-user), and the session `id` itself (which is often generated automatically) [31]. These identifiers let you retrieve or continue the correct session later, for example to resume a chat where you left off.

Each `Session` contains two main things:

- **Event History (`session.events`):** a chronological list of all events that have occurred in the session – user messages, agent responses, tool calls and results, etc. [30]. This is the conversation transcript, which the agent can refer to (e.g. an LLM agent will usually incorporate the recent events into its prompt for context).
- **Session State (`session.state`):** a dictionary of key–value pairs for any ephemeral data the agent wants to remember during the conversation [30]. This is like the agent's scratchpad for the session, used to store intermediate results, flags, or any info that persists across turns in this dialogue. For example, an agent might store `{"user_name": "Alice"}` to remember how to address the user, or keep track of a multi-step workflow state (`{"booking_step": "payment_confirmed"}`) [32]. The state is **per session** – it's not shared across different conversations – and it resets when a new session starts (unless explicitly initialized).

Managing all these sessions and their data is the job of the **SessionService**. ADK's `BaseSessionService` defines an interface for session management, and ADK provides implementations you can choose based on your needs [33]:

- **InMemorySessionService:** Keeps sessions in memory (RAM) only [34]. This is fast and requires no external setup, but everything is lost if your process restarts. It's great for local development or ephemeral agents.
- **DatabaseSessionService:** Persists sessions to a database (SQL) for durability [35] [36]. This allows sessions to survive restarts and scale across multiple processes if they share the same DB. You provide a DB connection string (e.g. MySQL, PostgreSQL, or even SQLite) when initializing it [37].
- **VertexAiSessionService:** Uses Google Cloud's Vertex AI Agent Engine to store and manage sessions in the cloud [38] [39]. This is meant for production setups on GCP – it offloads session storage to a managed service and can integrate with other Vertex AI features. (It requires setting up a GCP project, a Vertex AI Reasoning Engine, etc.)

Regardless of the backend, the SessionService's **core responsibilities** include starting new sessions, retrieving existing ones, listing sessions (e.g. find all conversation threads for a user), and appending new events to a session [40] [41]. You typically do not manipulate Session objects directly; instead, you call methods like `create_session(app_name, user_id)` or the Runner does so on your behalf. When the Runner begins `runner.run_async(user_id, session_id, new_message)`, it will use the SessionService to **fetch the Session** for that `user_id` and `session_id` (or create one if it's a new conversation) [42] [21]. Then as each event comes through, the Runner calls `session_service.append_event(session, event)` to log it. The SessionService is what updates the in-memory Session object and also ensures the changes are saved to whatever storage is backing it (writing

to the DB, etc.) [24]. This includes updating the session's state: if an event carries a `state_delta` (a change to session state), the SessionService will merge that into the current `session.state` at append time [43]. In other words, **session state is updated via events** – you generally don't manually set `session.state[...]` in the middle of an agent run. Instead, the agent emits an event indicating a state update, and the SessionService applies it as part of appending that event [43] [24]. (One-time initialization of state is possible when creating a session, but ongoing updates should go through events to maintain a consistent history [44] [45].)

Session Lifecycle: A typical session starts when a user initiates a conversation (explicitly or implicitly creating a Session via the Runner). The session accumulates events and state with each turn. If the conversation ends or times out, you might use `session_service.delete_session(...)` to clean it up, though often sessions can be kept around to allow the user to come back later [46]. You can also **resume** a session by providing the same session ID and user ID on the next Runner call – the Runner will load the saved history so the agent continues with full context [47]. This is how, for example, you could have a long-running chat that persists between user sessions or across server restarts. SessionServices often provide a method to list active sessions for a user/app, which can be useful if you need to manage or display conversation history externally [48] [49].

One more concept is **Session State vs. Long-Term Memory**: The session `state` is short-term, specific to that conversation. ADK also provides an optional **MemoryService** interface for long-term or cross-session memory [50]. A MemoryService (like the provided `InMemoryMemoryService` or a cloud-based vector store) can be used to store and retrieve information that persists beyond a single session [51]. For example, if you want your agent to "remember" facts a user mentioned in past conversations, or to have a knowledge base, you'd use a MemoryService. The Runner can be configured with a MemoryService (passed in similarly to session_service) and the agent can query it as needed. In practice, session state and memory serve different scopes: **state** is the working memory *per conversation*, and **memory** (via MemoryService) is a broader knowledge base that transcends one conversation (or is shared among many sessions if you choose) [51]. The MemoryService isn't heavily involved in the core event loop – it's more accessed by the agent logic (e.g., an agent might have a tool to do a semantic lookup in memory, or an LLM agent might be configured to auto-query the memory) – so we won't dive deeply here. Just know it's another service similar to SessionService in how it's plugged in, but serving a different purpose (long-term knowledge).

ArtifactService and Artifacts

In complex agent workflows, often you'll deal with **artifacts** – binary or large data pieces like images, audio files, PDFs, or any non-text content that an agent might need to consume or produce. ADK handles these via the **ArtifactService**, ensuring such data is stored and retrieved properly rather than mixing raw binary data into the text event stream. The `BaseArtifactService` defines how to save and load artifacts, and ADK provides implementations like:

- **InMemoryArtifactService:** Keeps artifacts in memory (again, volatile and for testing).
- **GcsArtifactService:** Saves artifacts to a Google Cloud Storage (GCS) bucket [52]. You configure it with a bucket name, and it will handle writing files to that bucket behind the scenes.

When the Runner receives a new user message with an attached file (say the user uploaded an image), it can use the ArtifactService to **save that file** and will typically put a reference (like a filename or ID) into the Event so the agent can access it if needed [9]. Similarly, if an agent generates an output that is a file or any

binary data (perhaps a tool that returns an image or a PDF), the agent can include that data as an artifact to be saved. In both cases, the artifact itself is stored (e.g., as an object in GCS or in-memory store) and the event carries an `artifact_delta` describing the artifact (for example, its filename or key and maybe a MIME type) [43]. The Runner, upon processing that event, calls `artifact_service.save_artifact(...)` with the given info and then confirms the artifact was saved by updating the session (the `artifact_delta` gets recorded so you know that artifact exists for this session) [24]. Later on, if the agent or another tool needs that artifact, it can call `artifact_service.load_artifact(...)` using the same keys to retrieve the content [53] [54]. Each artifact is versioned (the ArtifactService can keep multiple versions of the same filename, returning a revision ID when saving) [55] [56]. This is useful if, say, an agent updates a file in multiple steps; you can choose to retrieve the latest or a specific older version.

In summary, the **ArtifactService** is the piece that **handles large or binary data storage**, separate from the main session event log. It's optional – if your agent doesn't use any such data, you might not need an ArtifactService (Runner accepts `artifact_service=None`). But for many real applications it's crucial. For instance, if you have an agent that processes user-uploaded documents, you'd use ArtifactService to store those documents (perhaps in cloud storage) and then the agent's events would just reference the doc. This keeps the session history lightweight and the storage of blobs scalable. The SessionService and ArtifactService work hand-in-hand: the SessionService keeps track of artifact references (filenames, etc., usually stored as part of session state or events), while the ArtifactService deals with the actual blob content [24] [57].

Events: The Building Blocks of Execution

Events are the fundamental units of communication and action in ADK. Every meaningful thing that happens during an agent's run is captured as an Event [58]. This includes: a user's message coming in, the agent's reply (text) going out, the agent deciding to invoke a tool, the result coming back from a tool, any updates to state or memory, and even internal control signals. An `Event` in ADK extends the basic concept of an LLM response to include metadata and an **actions** payload for side effects [59] [60]. Key fields of an event include:

- **author:** who generated the event. This is `"user"` for user messages, or the name of the agent/tool that produced it for others (e.g. `"MyAgent"` or `"WeatherTool"`) [61] [62].
- **content:** the primary content of the event. This can be conversational text (with potentially multiple parts, e.g., if an LLM returned multiple segments or function call info), or it can represent a function call/request, or a function result. Under the hood ADK uses a structure compatible with the GenAI API, where `event.content.parts` might contain text or a `function_call` object, etc. [63] [64]. For example, if the agent's LLM decides to call a tool, it yields an event whose content contains a function call (with a tool name and arguments) – ADK marks this as a special event carrying a **Tool Call Request** [64]. After execution, the tool's result is wrapped in another event as a **Tool Result** (function response) [64]. If it's just regular conversation text, the content will have a text part. ADK also flags whether content is partial (streaming) or complete via `event.partial` [65].
- **id and timestamps:** each event has a unique ID and a timestamp, which are used to track ordering and for referencing events if needed [61].

- **actions:** this is a special field (an `EventActions` object) that carries **side-effect instructions** for the Runner and services [61]. This is where an event can say "update the session state" or "save this artifact" or "switch to another agent," etc. Common action fields include:
- `state_delta`: a dictionary of changes to apply to `session.state` [43]. For example, after a tool returns some info, an agent might include `state_delta={"latest_weather": "sunny"}` to record it. The SessionService will merge this into the current state when appending the event.
- `artifact_delta`: details of any artifact to save (e.g. `{ "save": {"filename": "image1.png", ...} }`) [43]. The Runner will trigger ArtifactService to actually save the artifact and then note it in the session.
- Control signals like `transfer_to_agent`: this indicates the agent is handing off control to a different agent (multi-agent handoff), and `escalate`: indicating maybe to exit or escalate the conversation [66]. For instance, a **SequentialAgent** might yield an event with `transfer_to_agent` to start running the next sub-agent in sequence, or an agent might yield an `escalate` event to signal some condition met (like ending a loop or indicating it cannot handle the request).
- There are other metadata in actions (like markers for end-of-turn, or listing any long-running tool tasks), but the above are key for understanding state and artifacts.

During execution, **the Runner and SessionService rely on events to know what to do**. When the Runner receives an event from the agent, it checks the event's `actions`. If there's a `state_delta`, it calls the SessionService to apply those updates to the session's state [24]. If there's an `artifact_delta`, it likely invokes the ArtifactService to save or load something and confirms that in the session record [24]. Only after these side-effects are handled does the Runner propagate the event outward (and let the agent continue) [15] [16]. This design means the agent code can "optimistically" say it wants to change state or save a file, and those changes become official only when the Runner processes the corresponding event. If an error occurred in processing (say the database was down and couldn't save session), the Runner could abort, and the agent wouldn't continue under false assumptions. In practice, this mechanism yields a robust sequence: event by event, with **all state and context consistently updated** as you go [23] [22].

It's also worth noting how **history and state** interplay with events. Because every interaction is an event, the complete `session.events` log is a detailed transcript – useful not just for the agent (to incorporate conversation history) but for developers to debug or audit the agent's behavior. You can inspect the events to see each tool call and result, each decision point, etc., which is invaluable for understanding complex agent chains [67]. Meanwhile, the `session.state` at any time is essentially a function of all the `state_delta` events up to that point. The SessionService often implements state updates as *appending special events* behind the scenes. Indeed, in ADK's design, **state changes are event-driven** – a direct consequence is that if you retrieve a session later, you can replay its events to reconstruct how the state came to be (which is great for reproducibility). Developers are generally advised not to manually mutate `session.state` during a run, but to use provided mechanisms (like an agent's `output_key` or tool results) that produce the appropriate events to change state [68] [69]. This ensures the changes are tracked.

To illustrate the event flow, imagine a simple scenario: the user asks *"What's the weather in Paris?"*. Your LLM agent might yield a **FunctionCall Event** asking to use a `get_weather` tool (with argument "Paris"). The Runner sees this and appends it to history [21] [70], then pauses the agent and actually executes the `get_weather` tool. Suppose the tool returns `{"temperature": "15°C", "conditions": "Cloudy"}`. The Runner will take that result and create a **FunctionResponse Event** (on behalf of the agent) containing the tool's output [71]. That event might have a `state_delta` – for example, updating `state["latest_weather"]` – and the Runner applies it (so now

session.state has `latest_weather` stored) [24]. The Runner then gives this event to the agent (resuming it), so the agent now "sees" the tool's result in context. Finally, the agent uses the info to produce a **Final Answer Event** with content like *"It's 15°C and cloudy in Paris."* which the Runner processes and delivers to the user [72]. All these steps are events in the session: the user question (by Runner, author user), the tool call request (by agent), the tool result (by agent, after Runner exec), and the final answer (by agent). The session state might now hold that latest_weather data, and if the user asks a follow-up, the agent can retrieve it from `ctx.session.state`.

In summary, **events are the lingua franca of ADK's runtime** [58]. They encapsulate communication between user, agents, and tools, and also carry the instructions for maintaining state and control flow [62] [66]. Understanding events is crucial because everything an agent does or decides translates into an event (or a series of events), which the Runner and services coordinate to make sure the whole system stays in sync.

Agent Types and Orchestration Patterns

ADK supports different types of agents and ways to compose them, enabling sophisticated orchestration of logic. At a high level, there are **LLM-powered agents** that use large language models to decide on actions, and **workflow agents** that deterministically organize other agents. Let's break down the main categories:

LLM Agents and Tool Use

An **LLM Agent** (in Python, `google.adk.agents.Agent` which is an alias of `LlmAgent`) is the standard agent type that wraps a Large Language Model to handle an assignment or task [73] [74]. This agent type uses prompts and the model's outputs to drive behavior. For example, you might configure an LLM agent with a system instruction like "You are a travel assistant" and give it access to tools like `SearchFlights` and `BookHotel`. When a user query comes in, the LLM will produce either a direct answer or a function call (tool invocation) based on the prompt and context.

LLM Agents are powerful because they can **reason dynamically** – they decide if and when to use tools, how to parse user requests, and what to output, all by prompting the model (no hard-coded logic for those decisions). In ADK, tools are integrated via OpenAI Function Calling-style semantics [75]. During setup, you attach a list of tools (functions) to the LLM agent. Each tool has a name and description (and parameter schema) that the LLM sees. When running, if the model's output includes a function call (e.g., it "decides" to call `get_weather(city="Paris")`), ADK will translate that into an Event (as described earlier) and the Runner will execute the tool. The tool is implemented as a normal Python function or method (or even another agent) – ADK provides a `FunctionTool` wrapper to turn a Python function into a tool that the agent can use [76] [77]. After execution, the result is passed back to the LLM agent as another event, and the agent's LLM can incorporate that result into its final answer [71] [78].

Tools dramatically extend what the agent can do: they allow it to query databases, call external APIs, perform calculations, etc. – essentially **grounding** the LLM in real actions and data [79] [80]. The ADK comes with a suite of built-in tools (for web search, code execution, document retrieval, etc.) and supports third-party integrations (like LangChain tools) [81] [82]. You can also easily define custom tools specific to your application (e.g., a function `def check_inventory(item): ...` and wrap it as a FunctionTool). The agent's instruction should guide the LLM on *when and how to use these tools*, and what to do with the results

[83] [84]. A good practice is to clearly enumerate the available tools and their purpose in the prompt, so the LLM knows its "capabilities." The ADK's design even allows chaining tools – for instance, the output of one tool can be fed into another – by virtue of how you instruct the LLM or by capturing outputs in state for later use [84].

In essence, an **LLM agent** is an intelligent, model-driven entity that can handle flexible tasks. It *uses* the ADK runtime to interact with tools and maintain context, but it decides its course of action via the model's outputs (so there's an element of nondeterminism and probabilistic decision-making).

However, when building complex systems, you might not want to leave the entire control flow to the LLM. That's where the next category comes in.

Workflow Agents: Sequential, Loop, Parallel

Workflow Agents are a special kind of agent in ADK whose sole job is to orchestrate other agents in a predefined manner [85]. They do **not** use an LLM for their logic – instead, they implement a fixed control structure (deterministic) like running things in order, or looping, or parallelizing tasks. This gives you predictable execution patterns (no "creative" deviations by the model) for high-level flow, while still leveraging LLM agents for the individual reasoning steps when needed [86] [87]. ADK provides three core workflow agent types:

- **SequentialAgent:** Executes a list of sub-agents one after another in a fixed sequence [88]. Use this when the order of operations is important and each step's output feeds the next. For example, if you need to **(1)** extract text from a PDF, then **(2)** summarize that text, you could have two sub-agents (perhaps an ExtractionAgent and a SummarizerAgent) and wrap them in a SequentialAgent so that step 2 only runs after step 1 completes. The SequentialAgent will simply call the first sub-agent, stream all its events to completion, then call the second, and so on [89] [90]. The key point: **strict order, no skipping**. This agent guarantees that sub_agents[0] finishes before sub_agents[1] starts, etc. [91] [92]. All sub-agents share the same session context by default (since they're under the same parent Runner invocation), so later agents can see the earlier agents' outputs (often via session state or event history). *Note:* A sequential workflow doesn't inherently have conditional logic; it will run all sub-agents in the given order every time, unless you build a condition into a sub-agent itself or use a custom agent for conditional steps (discussed later).

- **LoopAgent:** Repeats execution of its sub-agents either for a specified number of iterations or until a certain condition is met [93]. This is useful for iterative refinement tasks – for example, an agent that keeps refining a piece of text until it meets some criteria, or repeatedly attempts an action until success. The LoopAgent will execute its sub-agents in order (like a sequence) for one iteration, then before starting the next iteration, you (the developer) must decide whether to continue or stop. **Important:** the LoopAgent itself doesn't magically know when to stop – you must provide a mechanism for the termination condition [94]. In practice, this could be done by one of the sub-agents outputting a flag in the session state (e.g., setting `state["loop_done"] = True` when criteria are met) or by using a special tool/event (ADK includes an `ExitLoopTool` that an agent can call to break the loop). The LoopAgent will check for your condition at the end of each cycle. For example, if you configure it to run until `state["loop_done"] == True`, it will stop when that appears in state. Otherwise, it will start the next iteration, potentially using the updated state from the last round. This allows for things like *self-critique loops* (an agent keeps improving an answer until

a “satisfied” flag is true, or until a max iteration count is reached). Under the hood, a LoopAgent is implemented similarly to sequential execution on each iteration, but it encloses it in a Python loop. You, as the agent designer, ensure there’s progression or a breaking condition; otherwise, it could loop indefinitely [94].

- **ParallelAgent:** Runs multiple sub-agents concurrently (in parallel) [95]. This is great when you have several independent tasks that can be done at the same time – it can dramatically speed up workflows by utilizing concurrency. For instance, suppose a user asks for a comprehensive report that involves research on three different topics. Rather than doing one after the other, you could have three LLM sub-agents (each tasked with researching one topic) and put them under a ParallelAgent to execute simultaneously [96] [97]. The ParallelAgent will launch all its sub-agents’ `run_async` methods at roughly the same time (actually, it awaits them concurrently) [98]. Each sub-agent produces its own stream of events (its own chain of tool calls, responses, etc.), and the Runner intermixes handling of these events. It’s important to note that **by default there is no shared state or direct communication between parallel branches during execution** [98] [99]. Each sub-agent operates on the session independently at runtime – essentially each one sees the session as it was at start (plus any static differences you gave them, like different instructions). They don’t automatically see each other’s intermediate results until all are finished. This means if you need them to coordinate or if they produce outputs that must be combined, you typically do a follow-up step *after* the ParallelAgent. A common pattern is: run parallel sub-agents to gather info, each sub-agent writes its result to a unique `state` key (using an `output_key` so that its final answer is stored, say, under `"result_topic1"`, `"result_topic2"`, etc.), then use a SequentialAgent to feed into a next agent that merges those results [100] [101]. ADK’s documentation shows this pattern in a *Parallel Web Research* example: three research agents run in parallel, each storing their summary in state, then a synthesis agent (run after) reads those state entries to compose a final report [102] [103]. This gives you both speed and a combined outcome. If truly needed, one could implement some synchronization in parallel (like using a shared external store or locks), but often it’s simpler to keep parallel tasks independent and join results later [99] [104]. The ParallelAgent is deterministic in the sense that it will always spawn the same sub-agents; however, due to real concurrency, the order in which their events reach the Runner could vary slightly (though each sub-stream is ordered). The Runner will handle all, and you’ll get multiple event streams interleaved (the Runner yields events as they come from whichever branch is active). It’s up to your application to handle that if, for example, streaming to a UI (you might tag events by agent name when displaying).

A crucial aspect of workflow agents: they **do not perform any “thinking” with an LLM themselves** about how to route or modify the sequence – they simply execute the blueprint you’ve set. This makes them *predictable* and *reliable* for structuring complex workflows [86] [87]. Meanwhile, the actual “intelligence” of each step can still be powered by LLM Agents or other complex agents. You can even nest these: e.g., a SequentialAgent might contain a ParallelAgent as one of its sub-agents (for a phase of parallel work) and then another agent after that. Or a LoopAgent could have a complex chain inside it that it repeats. This composability lets you create fairly elaborate multi-agent systems without writing custom control code for common patterns [105].

Conditional Execution: One thing eagle-eyed readers might wonder is: How do I do a simple *if/else* in a workflow? For example, run AgentA, then if its result was X do AgentB, otherwise do AgentC. There isn’t a built-in “ConditionalAgent” class in the current ADK (2025) akin to sequential/parallel/loop. However, you can

achieve conditional logic using either the LLM itself or a **Custom Agent**. For simple cases, sometimes you let the LLM agent decide: e.g., an LLM could be instructed to decide which tool to call or which subtask to pursue based on user input (that's a form of learned conditional logic). But for explicit branching that you want to control in code (for determinism), you would use a CustomAgent.

Custom Agents for Complex Logic

A **Custom Agent** in ADK is essentially you writing your own agent class that inherits from `BaseAgent` and implementing your own `_run_async_impl` method (in Python) to generate events programmatically [106] [107]. This is the most powerful and flexible approach – you can write arbitrary Python logic to orchestrate sub-agents, implement loops or conditionals, integrate custom API calls, etc., all within this method. Of course, with great power comes complexity: you need to manage yielding events properly and ensure your logic aligns with ADK's async event loop model. But it unlocks any pattern you want that isn't covered by the standard workflow agents.

When to use a Custom Agent: The official guidance is to reach for a custom agent when your needs go beyond what Sequential/Loop/Parallel can do easily [108] [109]. Some examples include: - **Conditional branching logic:** e.g., *if condition, run AgentX, else run AgentY*. This is a prime use-case – rather than forcing an LLM to make the routing decision implicitly, you can code the condition and route to different sub-agents accordingly [110] [109]. - **Complex state management:** If you have to maintain intricate state across steps, or update multiple state variables in tandem, etc., a custom agent can explicitly manage `ctx.session.state` in code beyond simple sequential passing [111] [109]. - **External integrations at control flow level:** Suppose after an agent finishes you want to call an external API (not as a tool the LLM calls, but as a step in the workflow itself), or maybe log something to an external system, you can embed that in a custom agent's logic. - **Dynamic sub-agent selection:** Maybe you have a pool of sub-agents and based on input or intermediate results you want to pick one dynamically (not known ahead of time). Your custom agent could examine `ctx.session.state` or the content of events and then decide which agent to invoke next [109] [112]. - **Any unique pattern** that doesn't fit sequential, loop, or parallel precisely [112] – basically custom agents are your escape hatch to implement new orchestration patterns.

How to implement: In Python, you override `async def _run_async_impl(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]` [113]. This method will be called by the Runner (instead of the built-in LLM logic) when your agent runs. Inside, you have access to `ctx`, which includes `ctx.session` (with `.events` and `.state`), and references to the services if needed. You then write your control flow. Key things you'll do in custom `_run_async_impl`: - **Run sub-agents and yield their events:** You can define sub-agents as attributes of your custom agent class (for example, `self.step1_agent`, `self.step2_agent`). To execute them, call `async for event in self.step1_agent.run_async(ctx): yield event` [114] [115]. This will forward every event from the sub-agent up through your custom agent to the Runner. You can wrap this in conditional logic or loops as needed. For instance:

```
result = None
async for event in self.agentA.run_async(ctx):
    yield event
    # maybe capture final result event content
    if event.author == "agentA" and event.actions and event.actions.state_delta:
```

```
        result = ctx.session.state.get("some_key")
if result == "X":
    async for event in self.agentB.run_async(ctx):
        yield event
else:
    async for event in self.agentC.run_async(ctx):
        yield event
```

The above pseudo-code demonstrates capturing something from agentA's run and choosing B or C based on it. - **Read and write session state:** You can directly access `ctx.session.state` which is a Python dict [116] [117]. For example, `val = ctx.session.state.get("foo")` to read, or `ctx.session.state["bar"] = 123` to write. However, one **big caution**: if you write to session.state directly in your custom agent, that change is not automatically recorded as an event! It will exist in memory for the session, but if you want it to persist (in a database or to be recognized in history), you should emit an event with that state change. A common way is to have your sub-agents produce the state changes (via `output_key` or explicitly yielding a state update event). In a purely custom flow, you might manually yield an Event with a state_delta. But often it's easier to let built-in mechanisms handle it (for instance, an LLM sub-agent can be given an `output_key` parameter so that its final answer is automatically stored in session.state under that key [103] [118]).

That said, within a single invocation of a custom agent, you *can* use the state as a scratchpad freely. For example, set `ctx.session.state["working"] = True` and later check it; the user won't see that directly, and you could remove or override it before the invocation ends. (ADK even has a notion of **temporary state keys** prefixed with `temp:` that are automatically discarded at the end of an invocation [119] – ensuring they don't persist in the session history. This is advanced usage but mentionable: if you name a state key with `temp:`, the SessionService knows not to carry it over after the current run, treating it like a transient variable just for this chain of events.) - **Use normal Python control structures:** Inside `_run_async_impl` you can use `if/else`, `for` loops, `while` loops, `try/except` for error handling, etc., just like writing any async Python function [68] [120]. This means you can encode complex branching logic that would be cumbersome or unreliable to do with prompts alone. For example, you might loop until some condition in state is met (like implementing a custom loop with more checks than LoopAgent allows), or catch an exception from a tool call and decide to yield a custom error event, etc.

One thing to remember: your custom agent should **yield Events in a timely manner**. If you call a sub-agent and just wait for it to finish without yielding its events as they come, you might block the event loop. The pattern `async for event in sub_agent.run_async(ctx): yield event` ensures you stream sub-agent events upward immediately [114]. If you want to inspect or modify an event, you can do so between receiving and yielding it (e.g., log something or even change an event before yielding – though altering events is advanced and not usually needed).

Example use-case: Suppose you need an agent that first asks the user a question, then based on the answer either continues or ends. You could implement this as a custom agent that yields an event (prompting the user) and then stops – relying on the user's next input to resume the flow (this would actually be multiple turns). Or more directly, if the conditional decision can be made in one turn: run sub_agent1 (which perhaps sets `state["choice"]`), then check `ctx.session.state["choice"]` and

conditionally run sub_agent2 or sub_agent3. SequentialAgent alone would run both unconditionally, which isn't what you want.

While custom agents require writing code, ADK still integrates them into the same Runner and session system. So you run them via Runner like any agent, they yield events, etc. They are basically a way to implement your own mini-workflow agent if the provided ones don't suffice. The documentation even provides a full code example of a `StoryFlowAgent` that uses a custom logic to generate a story and then revise it (with loops and conditionals) [121] [122].

Multi-Agent Systems and Orchestration

Using the above building blocks, you can construct **multi-agent systems (MAS)** where different agents collaborate or pass control among each other. ADK is quite flexible in this regard: any BaseAgent can invoke another agent (for example, an LLM agent can *call another agent as a Tool*, via the special `AgentTool` wrapper, treating a sub-agent's capability as a function) [76] [123]. Also, as mentioned, workflow agents can contain LLM agents or even other workflow agents as sub-components.

For instance, you might have a **master agent** that is a SequentialAgent orchestrating a sequence of tasks: first a ParallelAgent to do some searches, then an LLM agent to summarize, then maybe a custom agent to decide if results are good or to trigger a loop for refinement. This hierarchical arrangement is managed through the same Runner – typically you set your root agent as this top-level orchestrator, and it will in turn run all the others as needed [124]. The events will include nested agent names (the `author` field will show which sub-agent produced each event, so you might see events from `ResearcherAgent1`, `ResearcherAgent2`, then from `SynthesisAgent`, etc., all in one session) [63] [125]. The `branch` field in Event (as seen in the Event object structure) can indicate hierarchical branches (like "which parallel branch this event came from") [126] [127], helping to keep track in complex scenarios.

Another feature for multi-agent orchestration is the **transfer of control** via events. ADK supports an `event.actions.transfer_to_agent` action, which essentially says: *end the current agent's run and hand off the remainder of this turn to another agent*. This is used under the hood by workflow agents, but you could also trigger it manually. For example, an LLM agent could decide "this query actually should be handled by a different specialized agent" – it could output an event with `transfer_to_agent = "OtherAgentName"` (and possibly some payload), and the Runner will swap in that other agent to continue the processing [66]. This is an advanced use-case (and requires that the Runner knows how to find or instantiate the target agent, usually configured in an AgentTeam or similar), but it shows that control can move between agents dynamically when needed. A more concrete scenario: imagine a triage agent that decides if a question is about weather, math, or chat, and then transfers to the appropriate expert agent – this could be done either with a custom agent that directly calls sub-agents (simpler), or with an LLM agent outputting a transfer signal (more on the fly).

Summary of Orchestration: With ADK's Python SDK, you have a toolbox of patterns: - Use **LLM Agents + Tools** for open-ended reasoning and external actions. - Use **SequentialAgent** to chain tasks in a defined order. - Use **LoopAgent** for iterative processes with a clear continuation condition. - Use **ParallelAgent** to fan out tasks concurrently and gather results faster. - Use **Custom Agents** when you need if-else logic, complex looping, or any bespoke control flow in code. - Combine them to build multi-step workflows (the output of one agent feeding into the next). All agents share the same **Session** context unless intentionally separated, which means you can pass information via the session state or events rather than through

function parameters. - Manage the whole execution with the **Runner**, which takes care of session management, event sequencing, and invoking these agents correctly.

When running *batch loads or multiple inputs*, you have a couple of options. If you want to process multiple independent inputs (with no shared context between them), you would typically create separate sessions for each (or simply call `runner.run_async` with different `session_id`/`user_id` each time). Each session will have its own state and history. You might even spin up multiple Runner instances if you need to isolate applications or use different configurations, but one Runner can handle multiple sessions (you always specify which session to use via user_id and session_id on each run call) [128] [129]. On the other hand, if you have a scenario where you want to feed multiple inputs sequentially *into the same session* (like a multi-turn conversation or a batch of messages forming one dialogue), you simply call `runner.run_async` repeatedly with the same session identifiers. The SessionService will keep appending events to the existing session, and the agent will see the growing history/state as context for each subsequent input [47] [130]. ADK is designed to handle continuous sessions like a chat, as well as one-off invocations.

One can also imagine using the ParallelAgent to process multiple inputs in parallel within one session context, but typically you'd do parallelization across sessions at a higher level (e.g., using Python asyncio or threads to call multiple runs concurrently), since parallel branches inside one session are more for logically parallel tasks of a single job rather than unrelated inputs. The design is flexible – what constitutes a "session" is up to your app's logic (it could be one user's conversation, or one task run, etc.).

Finally, all these features come together so that you can build an **"execution agent" with sophisticated orchestration** using ADK. You'll leverage the Runner to manage sessions and event flow, the SessionService to maintain context across turns or runs, the ArtifactService for any heavy data passing, and compose LLM agents with tools and workflow logic to achieve complex multi-step behaviors. The ADK Python SDK provides the classes and APIs (documented in the `google.adk` library) to do this programmatically – you instantiate agents (LLM or custom or workflows), combine them, and then execute them with Runner calls. Each run yields a stream of events that you can iterate over to observe the agent's inference chain in detail or to present output to users as it's generated [128] [131].

In summary, **Google ADK** gives you a structured yet flexible way to manage AI agent interactions: the Runner + SessionService ensure each session's **events** and **state** are tracked and persisted, while **agents** (LLM-based, tool-using, or workflow controllers) define *what* to do and *how* to orchestrate sub-tasks. By understanding these core pieces – Runner, events, session, state, artifact storage, and the various agent patterns – you can confidently build complex agent pipelines that handle everything from multi-turn dialogues to parallel tool-using workflows, all while maintaining the context and consistency needed for reliable AI behavior. **Everything is connected by the event-driven runtime**, which guarantees that your sophisticated orchestration executes in a clear, step-by-step fashion [19] [24], and that you can always inspect or intervene at the event level to tune or debug the system. With this knowledge, you're well-equipped to develop a new agent leveraging ADK's programmatic API for a wide range of use cases, orchestrating numerous components while the ADK infrastructure handles the heavy lifting of session management, state persistence, and event sequencing.

Sources:

- Google ADK Documentation – Sessions, Events, and Runtime [30] [43] [24]
- Google ADK Documentation – Workflow Agents (Sequential, Loop, Parallel) [92] [132]

- Google ADK Documentation – Custom Agents and Conditional Logic [110] [115]
- *Building Intelligent Agents with Google ADK* – Runner and Runtime deep-dive [8] [9]
- Google ADK API Reference (SessionService, ArtifactService, Runner) [133] [55]

---

[1] [2] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [42] [57] [70] [71] [72] [78] [119] Agent Runtime - Agent Development Kit
https://google.github.io/adk-docs/runtime/

[3] [4] [5] [6] [7] [8] [9] [10] [11] [26] [37] Chapter 16 - ADK Runner and Runtime Configuration | Amulya Bhatia
https://iamulya.one/posts/adk-runner-and-runtime-configuration/

[25] [27] [28] Runtime Config - Agent Development Kit
https://google.github.io/adk-docs/runtime/runconfig/

[29] [30] [31] [33] [34] [38] [39] [40] [41] [44] [45] [46] [47] [130] Session - Agent Development Kit
https://google.github.io/adk-docs/sessions/session/

[32] State - Agent Development Kit
https://google.github.io/adk-docs/sessions/state/

[35] [36] [48] [49] [52] [53] [54] [55] [56] [73] [74] [128] [129] [131] [133] Submodules - Agent Development Kit documentation
https://google.github.io/adk-docs/api-reference/python/google-adk.html

[43] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [125] [126] [127] Events - Agent Development Kit
https://google.github.io/adk-docs/events/

[50] [51] Memory - Agent Development Kit
https://google.github.io/adk-docs/sessions/memory/

[68] [69] [106] [107] [108] [109] [110] [111] [112] [113] [114] [115] [116] [117] [120] [121] [122] Custom agents - Agent Development Kit
https://google.github.io/adk-docs/agents/custom-agents/

[75] [76] [77] [79] [80] [81] [82] [83] [84] [123] Tools - Agent Development Kit
https://google.github.io/adk-docs/tools/

[85] [86] [87] [88] [105] Workflow Agents - Agent Development Kit
https://google.github.io/adk-docs/agents/workflow-agents/

[89] [90] [91] [92] Sequential agents - Agent Development Kit
https://google.github.io/adk-docs/agents/workflow-agents/sequential-agents/

[93] [94] [132] Loop agents - Agent Development Kit
https://google.github.io/adk-docs/agents/workflow-agents/loop-agents/

[95] [96] [97] [98] [99] [100] [101] [102] [103] [104] [118] [124] Parallel agents - Agent Development Kit
https://google.github.io/adk-docs/agents/workflow-agents/parallel-agents/